# Lecture Notes: TwinCAT 3

Structures and Functions

March 14, 2026

## Contents


**1 Introduction to Structures (DUTs)** **2**

**2 Functions (POUs)** **2**

**3 Passing Parameters: Value vs. Reference** **3**

**4 Practical Implementation: System Time & Libraries** **4**

4.1 Adding the Tc2_Utilities Library . . . . . 5

4.2 Complete Working Example . . . . . 6

**Quick Reference Card** **6**

# 1 Introduction to Structures (DUTs)

While **arrays** are used to hold multiple data items of the same kind, **structures** allow you to hold several data items of **different kinds**. Structures are user-defined data types (UDT) that represent a record or a logical group.

- **Advantages:** They are more user-friendly than maintaining multiple separate arrays for related data. If a data definition changes, you only need to update it in one place (the structure definition) rather than everywhere those variables are used.
- **Syntax:** To access components within a structure, use **dot notation** (e.g., `StructureName.ComponentName`).
- **Creation:** Right-click on the **DUTs** folder in the project tree, select **Add** > **DUT**, choose **Structure**, and give it a name (often prefixed with `ST_` by convention).
- **Scope:** In TwinCAT 3, structures have a **global scope**, meaning they are visible to all code within the project.

### Example: Structure Definition

```
// DUT Definition: ST_Event
// Created as a separate DUT file in the project tree under the DUTs folder
TYPE ST_Event :
STRUCT
    eEventType      : E_EventType;         // User-defined ENUM for event classification
    eEventSeverity  : TcEventSeverity;     // Beckhoff event severity type
    nIdentity       : UDINT;               // Unique identifier for the event
    sText           : STRING(255);         // Descriptive text for the event
    dtTimestamp     : DATE_AND_TIME;       // When the event occurred
END_STRUCT
END_TYPE
```

**Tip:** Use the `ST_` prefix convention for structure names to easily identify them throughout your codebase and distinguish them from other data types.

# 2 Functions (POUs)

Functions are essential for **modularization and code reuse**; you define the logic once and can use it many times throughout your software.

- **Components:** A function consists of a name, an optional **return type** (e.g., REAL, INT), **input parameters**, and **local variables** that act as a "scratch pad".
- **Returning Values:** To return a value from a function, you assign the result directly to the **function's name** within the code.
- **Advanced Features:**

- **Multiple Outputs:** Functions can be designed to return more than one value by defining additional output parameters.
- **VAR_IN_OUT:** This keyword allows variables to act as both inputs and outputs, meaning the function can both read from and write to the same variable.

### Example: Basic Function

```
// Function: Celsius_To_Fahrenheit
// Converts a temperature value from Celsius to Fahrenheit
FUNCTION Celsius_To_Fahrenheit : REAL
VAR_INPUT
    fCelsius : REAL;    // Input temperature in Celsius
END_VAR
VAR
    // Local variables (scratch pad) - none needed for this simple function
END_VAR

// Calculate and return the Fahrenheit value
// Assign the result to the function name to return it
Celsius_To_Fahrenheit := fCelsius * 1.8 + 32.0;
```

### Usage Example

```
// Calling the function from a program or function block
VAR
    fTempCelsius    : REAL := 25.0;
    fTempFahrenheit : REAL;
END_VAR

// Call the function and store the result
fTempFahrenheit := Celsius_To_Fahrenheit(fCelsius := fTempCelsius);
// Result: fTempFahrenheit = 77.0
```

## 3 Passing Parameters: Value vs. Reference

When sending data into a function, it can be passed in two ways:

- **Pass by Value:** The function receives a **copy** of the variable. Changes made inside the function do not affect the original data. This is best for small, simple data types like INT, REAL, or BOOL.
- **Pass by Reference:** The function receives the **memory location** of the original variable. Any changes made inside the function persist after the function completes. This is more efficient for large data sets like **structures** because it avoids the "memory penalty" of copying large amounts of data.
- **Implementation:** In TwinCAT, passing by reference is achieved using either the `VAR_IN_OUT` keyword or by defining a parameter as a `REFERENCE TO` a specific type.

| Aspect | Pass by Value | Pass by Reference |
|---|---|---|
| Data Passed | Copy of the variable | Memory address |
| Original Affected | No | Yes |
| Best For | Small types (INT, REAL, BOOL) | Large types (structures, arrays) |
| Memory Overhead | Higher (copy created) | Lower (no copy) |
| Keywords | `VAR_INPUT` | `VAR_IN_OUT`, `REFERENCE TO` |

Table 1: Comparison of Parameter Passing Methods

### Example: Passing by Reference

```
// Function: UpdateEventTimestamp
// Updates the timestamp of an event structure passed by reference
FUNCTION UpdateEventTimestamp : BOOL
VAR_INPUT
    stEvent : REFERENCE TO ST_Event;     // Reference to the event structure
END_VAR
VAR
    fbGetSystemTime : GETSYSTEMTIME;     // Function block instance for system time
    stFileTime      : T_FILETIME;        // Temporary storage for file time
END_VAR

// Get the current system time
fbGetSystemTime(timeLoDW => stFileTime.dwLowDateTime,
                timeHiDW => stFileTime.dwHighDateTime);

// Directly modify the referenced structure's timestamp member
// This change persists after the function returns
stEvent.dtTimestamp := FILETIME_TO_DT(stFileTime);

// Return TRUE to indicate successful update
UpdateEventTimestamp := TRUE;
```

⚠ **Warning:** When passing by reference, any modifications made to the parameter inside the function will directly affect the original variable. Ensure this is the intended behavior before using `REFERENCE TO` or `VAR_IN_OUT`.

## 4 Practical Implementation: System Time & Libraries

The sources illustrate these concepts through a practical example: a function that updates an event's timestamp using the PLC's system time.

- **External Libraries:** To access advanced functionality like system time, you may need to add references to libraries like **Tc2_Utilities**.
- **Format Conversion:** The system time is often provided as a 64-bit integer (FILETIME) which must be converted into a human-readable `DATE_AND_TIME` format using built-in functions like `FILETIME_TO_DT`.

- **Function Blocks (Preview):** To get the system time, the example uses a **Function Block** (`GETSYSTEMTIME`), which must be **instantiated** in the local variables before use.

### Example: System Time Conversion

```
// Declaration section
VAR
    fbGetSystemTime : GETSYSTEMTIME;     // Instance of the system time function block
    stFileTime      : T_FILETIME;        // Structure to hold the raw file time
    dtCurrentTime   : DATE_AND_TIME;     // Human-readable date and time result
END_VAR

// Implementation section
// Step 1: Call the GETSYSTEMTIME function block to retrieve system time
// The function block outputs the time as two 32-bit values (low and high DWORD)
fbGetSystemTime(
    timeLoDW => stFileTime.dwLowDateTime,   // Lower 32 bits of file time
    timeHiDW => stFileTime.dwHighDateTime   // Upper 32 bits of file time
);

// Step 2: Convert the T_FILETIME structure to a DATE_AND_TIME value
// FILETIME_TO_DT is a built-in conversion function from Tc2_Utilities
dtCurrentTime := FILETIME_TO_DT(stFileTime);

// dtCurrentTime now contains the current date and time in a readable format
// Example value: DT#2024-06-15-14:30:45
```

## 4.1 Adding the Tc2_Utilities Library

To use the `GETSYSTEMTIME` function block and `FILETIME_TO_DT` function, you must first add the library reference:

1. In the Solution Explorer, expand your PLC project
2. Right-click on **References**
3. Select **Add library...**
4. Search for **Tc2_Utilities** and add it

> **Tip:** The `T_FILETIME` structure represents time as the number of 100-nanosecond intervals since January 1, 1601. The `FILETIME_TO_DT` function handles this conversion automatically, making it easy to work with human-readable timestamps.

### 4.2 Complete Working Example

Below is a complete example combining all the concepts from this lecture:

```
// Function: LogEvent
// Creates and logs an event with the current system timestamp
FUNCTION LogEvent : BOOL
VAR_INPUT
    stEvent       : REFERENCE TO ST_Event;    // Event to be logged (by reference)
    eType         : E_EventType;              // Type of event
    eSeverity     : TcEventSeverity;          // Severity level
    nId           : UDINT;                    // Event identifier
    sDescription : STRING(255);               // Event description
END_VAR
VAR
    fbGetSystemTime : GETSYSTEMTIME;          // System time function block instance
    stFileTime       : T_FILETIME;            // Temporary file time storage
END_VAR

// Populate the event structure members
stEvent.eEventType      := eType;
stEvent.eEventSeverity := eSeverity;
stEvent.nIdentity       := nId;
stEvent.sText           := sDescription;

// Get and set the current timestamp
fbGetSystemTime(timeLoDW => stFileTime.dwLowDateTime,
                timeHiDW => stFileTime.dwHighDateTime);
stEvent.dtTimestamp := FILETIME_TO_DT(stFileTime);

// Return success
LogEvent := TRUE;
```

## Quick Reference Card

### Structure Declaration Syntax

```
TYPE <StructureName> :
STRUCT
    <member1> : <DataType1>;
    <member2> : <DataType2>;
    // ... additional members
END_STRUCT
END_TYPE
```

### Function Declaration Syntax

```
FUNCTION <FunctionName> : <ReturnType>
VAR_INPUT
    <inputParam> : <DataType>;              // Pass by value
    <refParam>   : REFERENCE TO <DataType>; // Pass by reference
END_VAR
VAR_IN_OUT
```

```
    <inOutParam> : <DataType>;                // Pass by reference (read/write)
END_VAR
VAR
    <localVar> : <DataType>;                  // Local scratch variables
END_VAR

// Function body
<FunctionName> := <returnValue>;              // Assign return value
```

### Common Naming Conventions

| Prefix | Type | Example |
|---|---|---|
| `ST_` | Structure | `ST_Event`, `ST_MotorData` |
| `E_` | Enumeration | `E_EventType`, `E_State` |
| `fb` | Function Block Instance | `fbGetSystemTime` |
| `st` | Structure Instance | `stEvent`, `stFileTime` |
| `dt` | DATE_AND_TIME | `dtTimestamp`, `dtCurrentTime` |

Table 2: Common TwinCAT Naming Conventions

*Last Updated: March 14, 2026*